Go / No-Go recommendation

NexaPay, release/1.4.0, before public exposure Samuel — 15 August 2026, v2 (revised after live verification)

Recommendation: No-Go

Not because the API is insecure. That turned out to be the test fixture. Because the transfer journey cannot be completed, the balance on screen does not match the ledger, and there is no environment in which any server-side rule can be checked.

Scope

I focused on the two journeys that can lose money, executing a transfer and holding a role-scoped session, plus the ledger an operator reads before approving one. Load, the settlement webhook, file upload and i18n were left out, with reasons given in the strategy. 42 automated tests across API, contract, consistency and E2E levels, plus a full live browser pass.

Results

42 tests: 9 passed, 9 failed, 24 skipped. Four Critical product defects, one Major, one Minor. Six further findings reclassified as environment artefacts. Of the five risks I scored at 20 or above, none has a passing test. The P1 gate does not pass.

The blocking issues

The transfer form does not submit. Valid form, no errors showing, button enabled, and clicking it fires no request, shows no toast and raises no error. I verified this by instrumenting `fetch` and `XMLHttpRequest`, and again with a native click. The journey the product exists for cannot be completed through the interface.

The balance does not match the ledger. The dashboard shows $48,291.00, the transactions page $50,563.80, and the 15 records sum to -$389.66. Two screens, two different numbers, $2,272.80 apart, neither reconstructable from the data. Income and Expenses did not move at all while 23 extra transactions sat in the database, so those two appear static. Operators approve payments against these figures.

The PIN is sent and stored in clear, against R08. It also cannot be fixed as specified, because the app runs over plain HTTP and `crypto.subtle` is therefore unavailable in the browser.

A correction I have to report

My first-round assessment rated the unauthenticated reads and the missing server-side validation as Blocker product defects. That attribution was wrong. The demo backend is a `json-server` fixture: `GET /db` returns the entire database, and `POST {"hello":"world"}` returns 201. A fixture like that has no auth and no validation by design, so six findings belong to the environment rather than to the backend team.

I also reported the balance as hard-coded. Also wrong. My test cycled the status filter and saw the number stay still, but filters change which rows are shown, not the account total, so that experiment could never have told the two explanations apart. What settled it was polluting the data by accident: the dashboard went to -$100,010,822.00 and back to $48,291.00 when I cleaned up. The figure is derived, just not from anything I can reconstruct.

One thing survives both corrections. Annexe H shows TC-11 and TC-12 failing on preprod, with an authenticated Member reading another user’s data. Preprod is not json-server and I have not seen it, so that concern stands, unevidenced either way.

Residual risk

Everything server-side is untested: idempotency, the amount ceiling, tenant scoping, contract conformance. Not skipped, but with nothing to test against. The settlement webhook has no coverage at all. No load validation, since Annexe I's figures were collected at roughly 0.55 requests a second against a 50/s target. Unit and component coverage remain invisible to me.

What has to be true for a Go

1. A QA environment running the real backend. Nothing below can be checked without it, so this gates everything else rather than being one item on a list.
2. The transfer journey completes end to end.
3. One balance, derived from the ledger, with the derivation written down.
4. HTTPS, and the PIN never sent or stored in clear.
5. Against the real backend: the 14 written transfer tests green, and Annexe H's TC-11 and TC-12 re-run and green.

After that, before public exposure: the Annexe E contract regenerated to match reality, `/health` and `/ready` exposed, and a load run at 50 req/s on a production-sized environment.

Two more weeks

Days 1 to 3, stand up a QA environment with the real backend, because everything else depends on it. Days 4 and 5, re-run the full suite there and re-attribute the six environment findings. Days 6 to 8, webhook coverage once someone gives me a signing secret. Days 9 and 10, the load suite. Days 11 and 12, consumer-driven contract tests so the Annexe E drift cannot happen again quietly. Days 13 and 14, an accessibility pass and a mutation-testing spike on `transfer-svc` to find out whether its unit tests assert anything.

In short

The front end is in worse shape than my first round suggested. Its most important button does nothing and its headline figure does not match the data behind it. The backend is not worse than I thought, it is simply not present in this environment, and I would push back on any release decision drawn from it.

Full evidence in `08-live-verification.md`.